NAME: B Sandhya

DATE:30-08-2026

REGNO:192311292

SUBJECT: CLOUD COMPUTING AND BIG DATA ANALYTICS

QUESTION 1 – Big Data Platform for Website Clicks, Mobile Interactions and Customer Transactions

Question

A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Answer

The described system handles **billions of events every day**, including website clicks, mobile interactions, and customer transactions. Such a workload cannot normally be handled efficiently by a single server. Therefore, the most probable design is a **distributed cloud-based big data architecture** consisting of distributed storage, event ingestion, batch and stream processing, analytical databases, and business intelligence tools.

1. Distributed Storage System

A distributed storage layer such as **Hadoop Distributed File System (HDFS)** or cloud object storage such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage is likely to be used.

The data can be divided into different zones:

- **Raw zone:** Original website and mobile events.
- **Processed zone:** Cleaned and transformed data.
- **Curated zone:** Analysis-ready datasets.

Large datasets can be stored using columnar formats such as **Parquet or ORC**, which reduce storage requirements and improve analytical query performance.

2. Data Ingestion Framework

A distributed messaging platform such as **Apache Kafka** is likely used to ingest continuous events.

For example:

Website → Mobile Application → Transaction Systems → Kafka → Data Lake

Kafka can receive events from multiple sources and distribute them across several partitions. This allows millions of events to be processed concurrently.

3. Batch Processing

Apache Spark is a probable batch-processing framework.

Spark can perform:

- Data cleaning
- Duplicate removal
- Data transformation
- Aggregation
- Customer behavior analysis
- Historical trend analysis

For example, Spark can process one month of website-click data and determine the most frequently visited products.

4. Structured and Semi-structured Data

Customer transactions are generally **structured data**, containing fields such as customer ID, product ID, transaction amount, and transaction date.

Website clicks and mobile events may be **semi-structured**, often represented using JSON.

The platform can store both types in a data lake while maintaining separate logical datasets.

Partitioning can be performed based on:

- Date
- Region
- Event type
- Customer ID

Indexes/catalogs can then help analytical engines locate the required data quickly.

5. Real-Time Stream Processing

For real-time analysis, **Apache Flink, Spark Structured Streaming, or Kafka Streams** could be used.

For example:

User Click → Kafka → Stream Processor → Real-Time Analytics → Dashboard

The system can calculate live metrics such as current website traffic, active users, product popularity, and transaction rates.

6. Scalability and Partitioning

Horizontal scaling is likely used because billions of events require additional processing capacity.

Kafka partitions distribute events across brokers, while Spark distributes computation across worker nodes.

Replication also provides fault tolerance. If one node fails, replicated data or another processing node can continue the workload.

7. Dashboards and Business Intelligence

After processing, aggregated data can be loaded into a cloud data warehouse or analytical database.

Business intelligence tools such as **Power BI, Tableau, or Grafana** can connect to this analytical layer.

Dashboards may display:

- Daily sales
- Website traffic
- Customer activity
- Conversion rates
- Popular products
- Regional performance

Architecture

Data Sources
↓
Kafka Ingestion Layer
↓
Distributed Data Lake / HDFS
↓
Spark Batch + Flink/Spark Streaming
↓
Data Warehouse / Analytical Database
↓
Power BI / Tableau / Grafana

Conclusion

The most likely architecture is a **distributed, scalable, fault-tolerant big data platform** using Kafka for ingestion, HDFS/cloud storage for large-scale storage, Spark for batch processing, and Flink or Spark Streaming for real-time analytics. Partitioning, replication, and horizontal scaling allow the platform to handle billions of events efficiently.

QUESTION 2 – E-Commerce Platform for Real-Time Personalization

Question

An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Answer

The e-commerce system continuously monitors customer behavior and requires immediate personalization. Therefore, it is likely based on a **real-time event-driven big data architecture** combining event streaming, NoSQL databases, caching, distributed processing, and machine learning.

1. Event Streaming

Apache Kafka is a probable event streaming platform.

Events can include:

- Product viewed
- Product added to cart
- Checkout abandoned
- Payment completed
- Search performed

Different Kafka topics can be created for different event categories.

For example:

Customer Activity → Kafka → Stream Processing

This allows customer activity to be processed almost immediately.

2. Customer Profiling Database

A scalable NoSQL database such as **MongoDB, Cassandra, or DynamoDB** is likely used to maintain customer profiles.

A customer profile may include:

- Customer ID
- Browsing history
- Purchase history

- Preferred categories
- Frequently viewed products
- Recent sessions

NoSQL databases are suitable because they can scale horizontally and provide fast access to frequently changing customer information.

3. Synchronization of Customer Data

Session data, clickstream events, and transactions can be synchronized using a common customer or session ID.

For example:

Product View → Session ID → Kafka → Customer Profile Update

When the customer completes a purchase, the transaction system can update the same profile.

This creates a continuously updated view of customer behavior.

4. Caching System

Redis is likely used as a high-speed caching layer.

It can store:

- Active sessions
- Shopping cart details
- Product information
- Personalized recommendations

Caching reduces repeated database queries and improves website response time.

5. Distributed Query Engine

A distributed query engine such as **Trino/Presto or Spark SQL** can query large datasets stored in the data lake.

This allows analysts to combine historical browsing behavior with transaction records.

6. Recommendation Machine Learning Workflow

The recommendation pipeline can be:

Customer Events → Data Cleaning → Feature Engineering → Model Training → Prediction → Recommendation

Machine learning models can use:

- Product views
- Previous purchases

- Cart additions
- Search history
- Product similarity

The trained model can generate personalized product recommendations.

7. Structured and Semi-structured Data

Order and payment information is generally structured, while clickstream information may be stored as JSON.

These datasets can be joined using:

- Customer ID
- Product ID
- Session ID
- Timestamp

Spark or another distributed processing framework can perform these large-scale joins.

8. Scalability and Fault Tolerance

Kafka partitions distribute events among brokers. NoSQL databases can use replication and sharding.

The processing system can automatically add computing resources when workload increases.

Checkpointing can be used in stream processing so that processing can recover after failures.

9. KPI Dashboards

Aggregated data can be sent to a warehouse or analytical database.

BI dashboards can monitor:

- Revenue
- Conversion rate
- Cart abandonment
- Average order value
- Customer engagement
- Recommendation click-through rate

Architecture

Customer Activity
↓
Kafka
↓

Flink/Spark Streaming
↓
Redis + Customer Profile Database
↓
Recommendation ML Model
↓
Personalized E-Commerce Website

Historical data can simultaneously flow into:

Data Lake → Spark/Trino → Analytics → BI Dashboard

Conclusion

The platform most likely uses Kafka, NoSQL, Redis, Spark/Flink, distributed querying, and machine learning to provide real-time personalization. Partitioning, replication, caching, and distributed processing provide scalability and fault tolerance.

QUESTION 3 – Big Data Healthcare System

Question

A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Answer

Healthcare systems generate highly diverse data, including structured patient records, medical images, wearable sensor readings, and prescriptions. Therefore, the most probable architecture is a **hybrid healthcare big data platform** combining relational databases, NoSQL systems, secure data lakes, streaming platforms, and machine learning.

1. Hybrid Database Architecture

Structured healthcare information such as patient IDs, prescriptions, laboratory results, and appointment details can be stored in a **relational database** such as PostgreSQL or MySQL.

Unstructured information such as diagnostic images and medical documents requires scalable object storage or a healthcare data lake.

Wearable sensor data can be stored in a **NoSQL or time-series database** because it generates continuous readings.

2. Data Integration

Hospitals, laboratories, and insurance systems may use integration tools such as:

- Apache NiFi
- Apache Kafka
- Cloud integration services

Healthcare interoperability standards such as **HL7 and FHIR** can help systems exchange patient information in a standardized format.

3. Streaming Healthcare Data

Wearable devices continuously generate sensor readings.

A probable architecture is:

Wearable Device → Kafka → Spark Streaming/Flink → Healthcare Analytics

The system can identify unusual patterns and generate alerts for authorized healthcare personnel.

4. Machine Learning

Machine learning can analyze patient histories and identify disease-risk patterns.

A typical workflow is:

Patient Data → Cleaning → Feature Engineering → Model Training → Risk Prediction → Clinical Decision Support

Models may analyze laboratory values, patient history, prescriptions, and other relevant information.

5. Privacy and Encryption

Healthcare information is sensitive, so security must be integrated throughout the architecture.

Likely controls include:

- Encryption at rest
- Encryption during transmission
- Strong authentication
- Role-based access control
- Data masking
- Secure backups
- Audit logging

For example, a doctor should only access patient information necessary for their authorized work.

6. Compliance

The platform should follow applicable healthcare privacy and data-protection regulations.

Every access to sensitive records should be logged. This allows administrators to identify unauthorized access or suspicious activity.

7. Analytics Pipeline

Historical data can be processed using Spark.

The analytics pipeline can be:

Healthcare Sources → Integration → Secure Data Lake → Data Processing → Analytics → ML → Clinical Dashboard

Population-level analysis can identify:

- Disease trends
- Treatment outcomes
- Medication patterns
- Readmission trends
- Public health patterns

Architecture

Hospitals + Labs + Insurance + Wearables
↓
HL7/FHIR + NiFi/Kafka
↓
Relational DB + NoSQL + Secure Data Lake
↓
Spark/Flink
↓
Machine Learning + Analytics
↓
Clinical Decision Support + Population Health Dashboard

Conclusion

A hybrid architecture is most appropriate because healthcare data varies greatly in structure and size. Distributed processing provides scalability, while encryption, access control, auditing, and interoperability mechanisms protect sensitive information.

QUESTION 4 – Smart City Big Data Platform

Question

A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Answer

A smart city generates continuous data from sensors, cameras, transportation systems, and weather services. Because the data is both high-volume and time-sensitive, the likely solution is an **edge-enabled distributed big data architecture**.

1. Edge Computing

Edge devices can process data close to where it is generated.

For example, traffic cameras can perform initial processing to determine:

- Vehicle counts
- Traffic density
- Average speed
- Possible incidents

Only useful metadata needs to be transmitted to the central platform, reducing network bandwidth.

2. Distributed Messaging

Apache Kafka is a likely messaging system.

Separate topics can be maintained for:

- Traffic sensors
- CCTV events
- Weather
- Public transport

Kafka partitions allow multiple consumers to process different data streams simultaneously.

3. Centralized Storage

A distributed data lake or cloud object storage system can store historical smart-city data.

Raw data can be retained for future analysis, while processed data can be stored in optimized analytical formats.

4. Geospatial Analytics

Traffic data contains geographical coordinates.

A geospatial analytics system can combine:

Location + Time + Traffic Density + Weather + Transport Data

This can identify areas experiencing congestion.

A spatial database such as **PostGIS** can support location-based queries.

5. Time-Series Database

Smart-city sensors generate continuous time-based readings.

A time-series database such as **InfluxDB or TimescaleDB** can store these measurements efficiently.

For example, the system can query traffic volume on a particular road over the previous 24 hours.

6. Congestion Prediction

Historical and real-time data can be processed using Spark or Flink.

A machine learning pipeline can be:

Historical Traffic → Feature Engineering → ML Model → Prediction

Real-time sensor information can then be combined with the model to predict congestion.

7. Batch and Real-Time Analytics

Real-time analytics can detect immediate traffic problems.

Batch analytics can identify long-term trends for:

- Road expansion
- Public transport planning
- Traffic signal optimization
- Infrastructure development

Therefore, both processing models are important.

8. Security and Access Control

The platform may contain sensitive infrastructure and citizen-related information.

Security controls can include:

- Encryption
- Authentication
- Role-based access control
- API security

- Network segmentation
- Audit logging
- Secure device communication

Only authorized departments should access sensitive infrastructure information.

9. Visualization

Tools such as **Grafana, Tableau, or Power BI** can display live information.

A smart-city control room could monitor:

- Current traffic
- Road incidents
- Public transport activity
- Weather conditions
- Predicted congestion

Architecture

Sensors / CCTV / Weather / Transport
↓
Edge Computing
↓
Kafka
↓
Flink/Spark Streaming
↓
Real-Time Dashboard

Historical data:

Data Lake → Spark Batch Processing → Geospatial + ML Analytics → Planning Dashboard

Conclusion

The smart city platform is likely designed using edge computing, Kafka, distributed storage, streaming analytics, geospatial databases, and machine learning. Combining real-time and batch analytics enables both immediate operational decisions and long-term urban planning.

QUESTION 5 – Healthcare Analytics System

Question

A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture

handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Answer

The described healthcare analytics platform combines information from multiple healthcare sources. Since the data includes sensitive patient information and continuous wearable readings, a **secure hybrid big data architecture** is likely to be implemented.

1. Hybrid Storage Architecture

Patient records, laboratory reports, and appointment details are mainly structured data and can be stored in relational databases.

Wearable readings are high-volume and semi-structured, making **NoSQL or time-series databases** suitable.

Medical documents and other large files can be stored in a secure cloud data lake or object storage.

2. ETL Pipeline

The platform requires an ETL process to combine data from hospitals and clinics.

The pipeline is:

Extract → Transform → Validate → Clean → Load

During transformation, inconsistent formats can be standardized.

For example, patient information from different hospitals can be mapped into a common format.

3. Interoperability Standards

Healthcare systems may use **HL7 and FHIR** standards to exchange information.

These standards improve interoperability between hospitals, clinics, laboratories, and other healthcare applications.

4. Real-Time Monitoring

Wearable devices can continuously generate readings.

A streaming platform such as Kafka can ingest these events.

Wearable → Kafka → Spark Streaming/Flink → Real-Time Analysis → Alert

This allows the platform to process incoming information without waiting for a large batch-processing cycle.

5. Predictive Health Analytics

Machine learning models can analyze historical patient information to identify risk patterns.

For example, models can generate risk scores based on relevant clinical and historical features.

The workflow is:

Patient Data → Data Preparation → Feature Engineering → ML Model → Risk Score → Clinical Decision Support

The results should assist qualified healthcare professionals rather than independently determine treatment.

6. Distributed Processing

Apache Spark can process millions of patient records across multiple computing nodes.

It can support population-level studies such as:

- Disease trends
- Treatment outcomes
- Readmission patterns
- Risk factors
- Healthcare utilization

Distributed processing makes large-scale analysis faster than processing the entire dataset on a single machine.

7. Encryption and Data Governance

Healthcare data requires strong security.

Likely mechanisms include:

- Encryption at rest
- Encryption in transit
- Role-based access control
- Multi-factor authentication
- Data masking
- Secure backups
- Audit logs

Access logs can record who accessed patient information and when.

8. Compliance

The platform should follow relevant healthcare and data-protection regulations applicable to the organization.

Data governance policies should define:

- Who can access information
- How long data is retained
- How data is shared
- How security incidents are recorded

9. Machine Learning Applications

Machine learning can support:

- Diagnosis risk scoring
- Disease prediction
- Patient deterioration prediction
- Treatment outcome analysis
- Personalized clinical decision support

Models should be validated and monitored to reduce errors and ensure reliable results.

Architecture

Hospitals / Clinics / Labs / Wearables
↓
ETL + HL7/FHIR Integration
↓
Secure Data Lake + Relational DB + NoSQL/Time-Series DB
↓
Kafka → Spark/Flink
↓
ML + Population Analytics
↓
Clinical Decision Support + Healthcare Dashboards

Conclusion

The most probable solution is a hybrid and secure healthcare analytics platform combining relational databases, NoSQL/time-series storage, ETL, HL7/FHIR interoperability, Kafka, Spark/Flink, and machine learning. Encryption, access control, auditing, and governance are essential for protecting sensitive healthcare information.